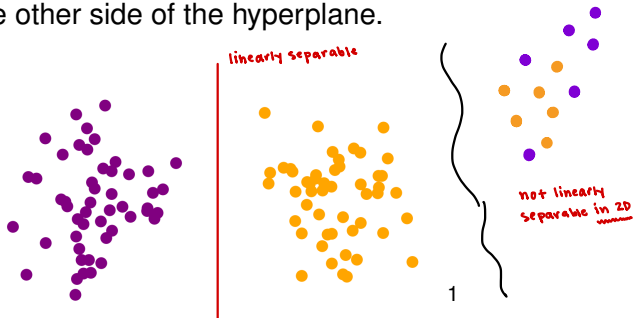


Week 8, Kernel methods
CAPP 30254, Machine Learning for Public Policy

MACHINE LEARNING
MACHINE LEARNING
FOR PUBLIC POLICY FOR
SPRING 2025
SPRING 2025

Linearly separable data

Recall that linearly separable data is data that can be separated by a hyperplane so that all points from one class lie on one side of the hyperplane and all points from the other class lie on the other side of the hyperplane.



¹Generated by GPT-4, DALL-E 3 after the prompt: “could you give another example of linearly separable data with purple and orange circle”

Algorithms and separable data

Some algorithms require linearly separable data:

- Perceptron
- Hard-margin SVM

Algorithms don't quite require linearly separable data:

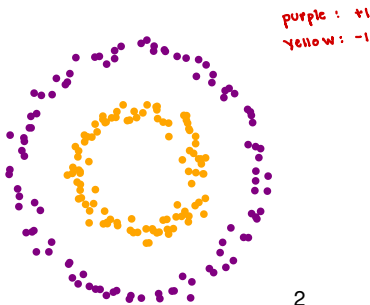
- Logistic regression
- Soft-margin SVM

Many algorithms don't require separability:

- Decision trees
- K-Nearest-Neighbors
- Neural networks

Non-linearly separable data

Some data that doesn't look linearly separable can be transformed so that it is separable in higher dimensions.

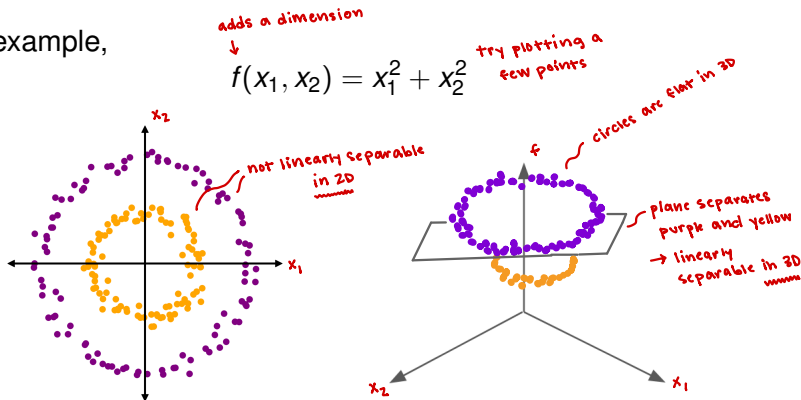


²Generated by GPT-4, DALL-E 3 after the prompt: "Can you draw circular non-linearly separable data set?"

Feature mapping

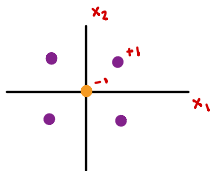
We can map features to a higher-dimensional space. Linear separators may exist in the transformed space.

For example,



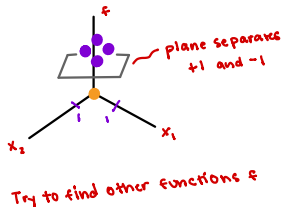
Feature mapping

Consider the following labeled data. First, identify which data point makes this set **not** linearly separable? Can you find a feature mapping so that the set *is linearly separable?*



x_1	x_2	label
-1	1	+1
1	1	+1
0	0	-1
1	-1	+1
-1	-1	+1

Ex. $f(x) = |x_1| + |x_2|$



Sketch the hyperplane that separates the positive and negative examples.

Kernel methods

That was the motivation —
We've been making kernel functions

Kernel methods formalize and generalize this idea.

Let $\mathbf{x}_i$ be a feature vector and map it:

previous examples:
 $f: \mathbb{R}^2 \rightarrow \mathbb{R}^3$

$$\mathbf{x}_i \in \mathbb{R}^{\overset{p}{\circ}} \rightarrow \phi(\mathbf{x}_i) \in \mathbb{R}^{\overset{d}{\circ}}$$

input space feature space

where $\mathbb{R}^d$ is a high dimensional space. We just saw some examples where $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3$. In practice $d \gg p$.

Our goal is to find a weight vector $\mathbf{w} \in \mathbb{R}^d$ so that $\hat{y}_i = \mathbf{w}^T \phi(\mathbf{x}_i)$ that gives good predictions.

linear in $\mathbb{R}^d$
* not linear in $\mathbb{R}^p$ *

Mapping is expensive

High-dimensional computations are costly. Is there a way to compute inner products in high-dimensions without performing the explicit mapping?

Kernel methods allow us to do this without computing $\phi(\mathbf{x}_i)$ explicitly for all training samples $\mathbf{x}_i$.

The Kernel Trick

Many machine learning models depend on dot products between examples to be computed. Kernel methods allow us to take dot products without computing $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ explicitly for all training samples $\mathbf{x}_i$. This is especially important because $d \rightarrow \infty$.

The kernel function is:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

The kernel trick lets us perform dot products as if we had transformed the data, without actually doing the transformation. This makes things a lot faster and we don't need to compute ϕ directly.

Popular kernels

- Linear kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$ no transformation
- Polynomial (degree q): $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j)^q$ features of degree q
- Gaussian (RBF): $K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{\sigma^2}\right)$ infinite dimensional - smooth boundary
- Sigmoid: $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\alpha \mathbf{x}_i^T \mathbf{x}_j + c)$ similar to neural networks

Polynomial (up to degree q): $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + c)^q$
polynomial up to degree q

Kernels and SVM

2D:



Recall that SVM chooses the decision boundary that maximizes the margin between two classes of data. It works perfectly when the data is linearly separable. The decision line is a linear function.

When the data isn't linearly separable, we can use a soft-margin SVM where some data points are allowed to be inside the margin or even allowed to be misclassified.

Another solution is to use an SVM transformation function $\phi(\mathbf{x}_i)$.

Ex. straight line in 5D
(linearly separable)

5D $\rightarrow$ 2D



nonlinear classifier!



3

³Generated by GPT-4, DALL-E 3 after the prompt: “Hi, could you please make a minimalist logo for a machine learning class...”